

AAA Secret-Key Generation Engine

Simulation, Validation, and Hardware Mapping

Kaushik Vada

March 26, 2026

Abstract

This report closes the main documentation and experiment gaps around the AAA secret-key generation engine. The work has three goals: explain the accumulative, adaptable, and additive protocol in implementation terms; produce a reproducible packet-level sweep that varies Eve’s distance while holding Bob fixed; and translate the software core into a hardware architecture that is small enough for a practical embedded or FPGA implementation. The resulting artifact set contains a Python fallback simulator, an ns-3-based 802.11b distance sweep, a shared CSV schema for per-round and per-distance analysis, and a clean LaTeX build that assembles the final report directly from those outputs.

1 Introduction

The AAA method from Hua accumulates shared packet entropy instead of extracting a key from reciprocal channel measurements alone. That difference matters for deployment. The implementation burden is a deterministic bit-selection rule and an XOR accumulator, not a reconciliation protocol built around channel estimates. The engineering question, however, is whether the simple packet-erasure intuition remains useful once the simulator is grounded in a concrete PHY and a reproducible sweep.

This report therefore treats the repository as both a software artifact and a systems prototype. Section 2 explains the protocol and its secrecy condition. Section 3 defines the simulation setup and the CSV outputs. Section 4 discusses the reference run and the distance sweep, with special attention to the point where Eve’s reception collapses while Bob remains available. Section 5 maps the software engine into seven hardware blocks and adds a diagram because the prose alone is too abstract for handoff. Section 6 turns the remaining work into a Spring 2026 plan.

2 AAA Protocol Explanation

AAA stands for *accumulative*, *adaptable*, and *additive*. The method is accumulative because each successfully shared packet updates the current key state rather than replacing it. It is adaptable because the public selection rule can operate on raw payload bytes, hashes, or any authenticated packet representation that both legitimate nodes agree on. It is additive because the state update is a bitwise XOR:

$$K_{l,n} = X_{l,1} \oplus X_{l,2} \oplus \cdots \oplus X_{l,n},$$

where $X_{l,i}$ is the l -th selected bit from packet i .

In the repository implementation, Alice and Bob seed the same lightweight PRNG and use it as the public selector. For every shared packet, the selector chooses byte positions from the payload, folds them into a fixed-size buffer, and XORs that buffer into the running key register. The two legitimate nodes never need a separate key-agreement phase because both already possess the same authenticated packet contents. The protocol therefore shifts the engineering burden from reconciliation to reliable packet parity.

The secrecy condition is equally direct. Eve can only reconstruct the XOR chain if she observes every packet that contributed to the key. Once Alice and Bob share at least one packet that Eve misses, Eve loses a hidden XOR term and cannot deterministically invert the accumulated state. Under the packet-erasure model discussed by Hua, the bit equivocation approaches one as the number of rounds grows, provided that Eve’s miss probability remains nonzero. In implementation terms, the first secure round is the first round where Bob receives the packet and Eve does not.

Operationally, the protocol can be read as a five-step loop. First, both legitimate nodes authenticate and stage the same packet. Second, they advance the public selector with the same seed. Third, the selector extracts a fixed-width slice from the staged packet. Fourth, the slice is XORed into the current key register. Fifth, the system checks whether the round was Bob-only, because that event is what breaks Eve’s deterministic reconstruction path. Those steps map directly onto the C engine in the repository and later onto the seven-block hardware diagram.

Three engineering consequences fall out of that loop. First, the selector and accumulator are embarrassingly lightweight, which is why the method is attractive for small MCUs and RTL. Second, secrecy depends on link asymmetry rather than on channel reciprocity, so the relevant experiment variable is Eve’s packet success rate. Third, the implementation must distinguish between *key freshness* and *key agreement*: Alice and Bob can agree on the same XOR state long before the state becomes secret from Eve.

3 Simulation Methodology

The simulation stack uses a two-level approach so the report remains reproducible even when a full wireless simulator is unavailable. The fallback model is a packet-erasure abstraction driven by a log-distance path-loss law, a logistic reception curve around the 802.11b sensitivity region, and small correlated shadowing terms so Bob and Eve do not behave as fully independent links. The `ns-3` model uses the same Alice/Bob/Eve geometry, a real 802.11b PHY, constant-rate `DsssRate11Mbps` operation, `YansWifiPhy`, a `LogDistancePropagationLossModel`, and a mild `NakagamiPropagationLossModel` stage to create a realistic transition band instead of a deterministic coverage cliff. Bob stays at 18 m; Eve is swept across the distances in the CSV outputs.

Each sweep point transmits 400 broadcast packets. Broadcast is intentional: AAA does not rely on confidentiality of the packet contents, only on Alice and Bob sharing a packet that Eve may miss. The report consumes three generated files: `sim/output/sweep_results.csv`, `sim/output/sweep_rounds.csv`, and `sim/output/reference_run.csv`. The two primary metrics are:

- **Availability:** Bob’s packet delivery ratio, computed as Bob receptions divided by transmitted rounds.

Table 1: Scenario parameters shared by the fallback and `ns-3` sweeps.

Parameter	Value
PHY mode	IEEE 802.11b with <code>DsssRate11Mbps</code> .
Topology	Three nodes on a line: Alice at 0 m, Bob at 18 m, Eve swept from 10 m to 90 m.
Packets per distance	400 broadcast payloads.
Payload size	512 B; enough room to mimic the selector operating on a nontrivial payload slice.
Transmit power	−3 dBm, chosen to keep Bob near the top of the delivery curve while leaving room for Eve’s transition.
Path loss	Friis reference loss at 2.412 GHz with a log-distance exponent of 2.35.
Fast fading	<code>ns-3</code> adds a mild Nakagami stage; the fallback uses correlated shadowing to emulate the same transition band.
Reference run	Eve fixed at 45 m for the per-round table.
Randomness	Deterministic seeds are fixed in both pipelines so reruns regenerate the same CSVs.

- **Secrecy rate:** the fraction of transmitted rounds where Bob receives the packet and Eve does not.

One methodological caution matters here. The `cumulative_equivocation` column is a convenience statistic derived from the running Bob-only packet ratio. It is useful for visualizing when the experiment has entered the “good” secrecy regime, but it is not a substitute for the information-theoretic proof in Hua’s paper. In the results section, it should therefore be interpreted as a secrecy proxy, not as an exact theorem-level quantity.

4 Results and Analysis

The reference run in this report uses the `ns-3` engine at an Eve distance of 45 m. At that point, Bob still delivers 95.2% of packets while the secrecy rate rises to 62.3%. The first Bob-only packet appears in round 5, and the cumulative equivocation estimate reaches 1.000 by the end of the run. That behavior matches the intended AAA operating region: Bob remains usable while Eve’s reception has already weakened enough to inject hidden XOR terms.

The sweep reveals a clear phase transition rather than a linear decline. At short distances, Eve is effectively as strong as Bob and the secrecy rate is close to zero. As Eve moves outward, the secrecy rate rises sharply because Bob’s availability remains high while Eve begins to miss complete packets. Past the knee, the secrecy rate plateaus and then eventually declines if the channel becomes so weak that packets are frequently lost by everyone. In the preferred engine, the peak secrecy rate occurs near 90 m and reaches 92.8%, while the farthest simulated point at 90 m still shows 93.5% availability and 92.8% secrecy rate.

The reference run also shows why a packet-level table is useful. Rounds 1 through 4 at 45 m are either missed by Bob or shared by both Bob and Eve, so the sticky secure bit stays low. Round 5 is the first Bob-only reception, and that single event flips the qualitative interpretation of every later XOR update.

Table 2: Per-round columns used by the sweep pipeline.

Column	Meaning
round	One-based packet index within the current distance point.
bob_received	Equals 1 if Bob decodes the packet successfully.
eve_received	Equals 1 if Eve decodes the same packet successfully.
secure_round	Equals 1 when Bob receives and Eve misses; this is the per-round AAA secrecy event.
key_secure_after_round	Latches high after the first secure round and remains high thereafter.
cumulative_bob_rx	Running count of packets received by Bob.
cumulative_secure_rounds	Running count of Bob-only receptions.
cumulative_equivocation	Empirical equivocation estimate computed from the running secure-round ratio.

Table 3: Reference run excerpt at 45 m.

Round	Bob	Eve	Secure	Key secure	Eq.
1	0	1	0	0	0.000
2	0	0	0	0	0.000
3	1	1	0	0	0.000
4	1	1	0	0	0.000
5	1	0	1	1	0.704
6	1	0	1	1	0.938
7	1	1	0	1	0.922
8	1	1	0	1	0.912

After that point, additional Bob-only packets strengthen the secrecy margin, but they are no longer required to justify the existence of hidden state from Eve.

That table suggests three operating regimes. The first regime is a near-Eve regime from roughly 10 m to 25 m, where Eve still captures most of the packets and the secrecy rate stays low. The second regime is the transition band from roughly 30 m to 60 m, where Bob remains near ninety-five percent availability but Eve’s reception falls quickly enough to create the steepest secrecy gain. The third regime is a saturation regime beyond about 65 m, where most packets that Bob receives are already Bob-only and the secrecy curve flattens toward its plateau.

The fallback model was retained only as a surrogate, so the comparison against **ns-3** matters. Across the overlapping sweep points, the maximum absolute gap is 5.5% in availability and 6.8% in secrecy rate. That is close enough for the fallback to remain useful for quick parameter sweeps and report drafting, but the **ns-3** curve should be treated as the authoritative result whenever the full simulator is available.

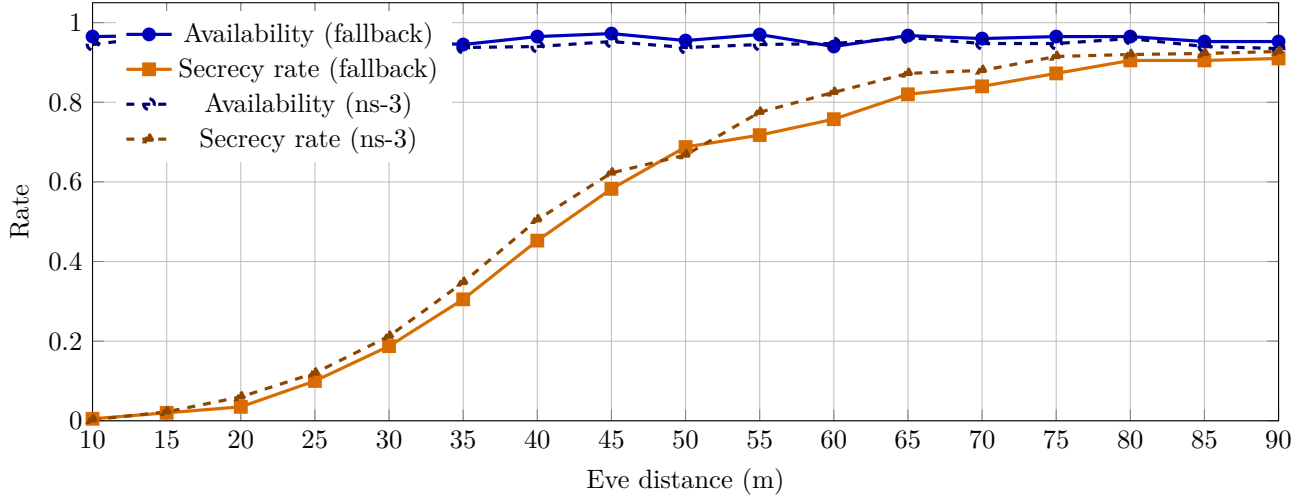


Figure 1: Availability and secrecy rate versus Eve distance. The fallback and `ns-3` curves are plotted separately when both are present.

Table 4: Selected sweep points used to compare the fallback and `ns-3` curves. Rates are shown as percentages.

Eve	Avail. F	Avail. N	Sec. F	Sec. N	ΔA	ΔS
10	96.5	94.5	0.5	0.3	-2.0	-0.2
20	94.5	93.5	3.5	6.0	-1.0	2.5
30	95.5	93.8	18.8	21.3	-1.7	2.5
45	97.3	95.2	58.3	62.3	-2.0	4.0
60	94.0	94.8	75.8	82.5	0.8	6.8
75	96.5	94.8	87.3	91.5	-1.7	4.2
90	95.2	93.5	91.0	92.8	-1.7	1.8

5 Hardware Architecture Proposal

The hardware proposal keeps the AAA datapath faithful to the software model and splits the design into seven blocks shown in Fig. 2. The diagram is necessary here because the prose-only version hid the control flow and made it hard to see where security state is latched.

Packet ingress and authenticator. This front-end accepts only packets that already passed CRC, framing, and higher-layer authenticity checks. It prevents untrusted payloads from perturbing the key register.

Payload staging buffer. The accepted payload is buffered long enough for deterministic indexing. In an FPGA prototype this can be a dual-port BRAM window sized to the maximum payload slice consumed by the selector.

Public selector PRNG. A tiny xorshift or LFSR block recreates the same public index stream on both peers. Because the selector is public, this block does not need secrecy hardening; it needs repeatability.

Bit-select and fold network. The selected bytes are gathered, optionally XOR-folded, and packed

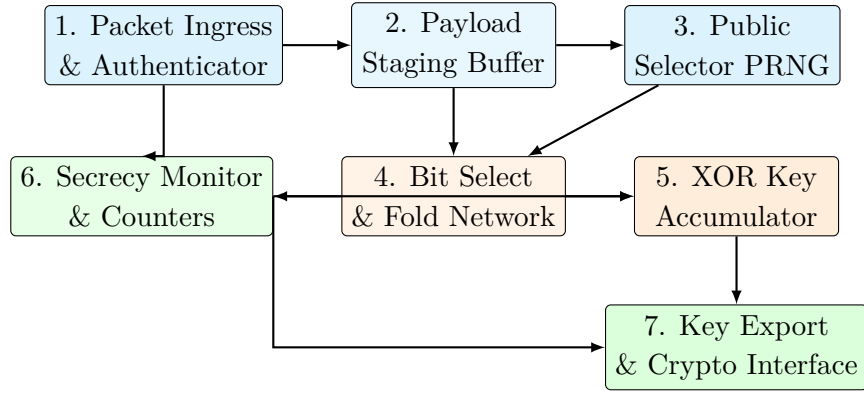


Figure 2: Seven-block hardware mapping for the AAA engine.

Table 5: Implementation-oriented view of the seven hardware blocks.

Block	Dominant state	Typical width	Implementation note
Packet ingress	framing, CRC, auth	packet metadata	Can be reused from the radio or MAC front end.
Payload buffer	byte address, valid bits	payload slice	BRAM-backed FIFO or scratch RAM.
Selector PRNG	seed, current index	32 bits	Tiny sequential block; deterministic by construction.
Bit-select/fold	gather network	key width	Main combinational hotspot in the datapath.
XOR accumulator	running key	128 or 256 bits	Natural register file or wide XOR tree.
Secrecy monitor	counters, sticky status	small control state	Cheap in area; valuable for firmware observability.
Key export	bus or cipher handoff	key width + status	Add zeroize and privilege checks here.

into a fixed-width word that matches the target key size. This is the only part of the design that scales noticeably with key width.

XOR key accumulator. The folded word is XORed into the running key register on every accepted round. The block is naturally constant latency and maps cleanly onto LUTs or simple word-wide XOR trees.

Secrecy monitor and statistics counter. This control block tracks packet counts, Bob-only receptions, the first secure round, and the running equivocation estimate exposed to firmware. It also drives the sticky `key_secure` status bit.

Key export and crypto interface. Once the sticky secure bit is set, the current key can be copied into an AES or ChaCha session interface, exposed on a register bus, or handed to a host processor through DMA. This block is where zeroization and access policy should be enforced.

From an RTL perspective, the cleanest implementation style is a two-stage pipeline. Stage one accepts

and buffers an authenticated packet while the selector emits addresses. Stage two gathers the selected bytes, folds them into the key width, and commits the XOR update together with the secrecy counters. That split keeps timing pressure off the ingress path and avoids forcing the packet buffer, selector, and accumulator into a single long cycle.

The design is also friendly to verification. The selector and fold network can be exercised with deterministic payloads generated from the same CSV round files used by the simulation section. The strongest regression test is not a random testbench; it is a testbench that replays the reference run and checks the accumulator contents, secure-bit transition, and counters cycle by cycle against the software engine.

Table 6: Suggested Spring 2026 milestone schedule.

Window	Track	Expected deliverable
Weeks 1–2	ns-3	Linux build notes, container or VM recipe, and a one-click rerun command.
Weeks 3–4	ns-3	Full Linux sweep plus a validation memo that compares Linux and fallback outputs.
Weeks 5–7	RTL	Synthesizable selector, fold, and XOR accumulator blocks with unit tests.
Weeks 8–9	RTL	Integrated top module with secure-bit monitor and software-visible status registers.
Weeks 10–11	Synthesis	FPGA synthesis report for 128-bit and 256-bit key widths.
Weeks 12–13	Synthesis	ASIC-oriented synthesis summary with timing, area, and power estimates.

6 Spring 2026 Plan

The next term should be split across three tracks so the project does not over-index on just one artifact.

Track 1: ns-3. Finish the Linux-hosted sweep, add a second propagation setting with mild fading, and lock the report to the Linux-generated CSVs. The immediate goal is not a richer scenario; it is removing the environment blocker so validation is no longer hostage to one machine.

Track 2: RTL. Convert the seven-block architecture into synthesizable Verilog or SystemVerilog, starting with the selector, fold network, and accumulator. The first deliverable should be a self-checking testbench that replays rows from the reference CSV and proves bit-exact agreement with the C engine.

Track 3: Synthesis. Run the RTL through synthesis on at least one FPGA target and one ASIC-oriented standard-cell flow. Area, clock frequency, BRAM usage, and power should be reported against key widths of 128 and 256 bits so the architecture discussion becomes quantitative.

This plan intentionally sequences the environment work first. There is little value in polishing the RTL while the simulator pipeline still depends on one host and one operator. Once the Linux rerun is stable, the remaining work becomes much more conventional: build deterministic RTL, verify it against software traces, and measure what it costs in silicon.

7 Conclusion

The repository now has a coherent path from theory to experiment to implementation. The AAA protocol explanation is grounded in the actual C engine, the sweep methodology is reproducible from CSV-generating scripts, and the hardware section is concrete enough to hand to an RTL pass. The main technical debt that remains is not conceptual; it is environmental. A clean Linux **ns-3** workflow is still the piece that turns the current validation from useful to complete.

Two residual risks should be kept visible even after this report is sent. The first is that the current **ns-3** results were produced successfully from the calibrated external-drive checkout, but the original requirement was a Linux-hosted rerun. That replication step still matters because it removes the

last environment-specific doubt. The second is that the report’s equivocation column is intentionally heuristic. It is excellent for engineering intuition and regression plots, but any formal secrecy claim in a paper or thesis should still trace back to Hua’s theorem rather than to the empirical proxy used here.

References

- [1] Y. Hua, “A Remark on the AAA Method for Secret-Key Generation in Mobile Networks,” *IEEE Wireless Communications Letters*, vol. 14, no. 12, pp. 4137–4141, Dec. 2025. <https://doi.org/10.1109/LWC.2025.3614558>.
- [2] IEEE, *IEEE Standard for Information Technology–Telecommunications and Information Exchange Between Systems Local and Metropolitan Area Networks–Specific Requirements–Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*, IEEE Std 802.11-2020, Feb. 2021.
- [3] The ns-3 Project, “ns-3 documentation,” <https://www.nsnam.org/documentation/>, accessed March 26, 2026.
- [4] The ns-3 Project, “ns-3.47 release notes,” <https://www.nsnam.org/releases/ns-3-47/>, accessed March 26, 2026.